# 52. Связи, область видимости и продолжительность жизни переменных

Мы уже ранее рассматривали, что такое область видимости, продолжительность жизни, связи и то, какими они могут быть в языке C++. Давайте сейчас закрепим это всё.

## Оглавление:


1. Область видимости
2. Продолжительность жизни
3. Связи
4. Резюмируем
5. Предварительные объявления


## Область видимости

**Область видимости** идентификатора определяет, где он доступен для использования. К идентификатору, который находится вне области видимости, доступ закрыт.

- Переменные с **локальной/блочной областью видимости** доступны только в пределах блока, в котором они объявлены. Это:
  - локальные переменные;
  - параметры функции.
- Переменные с **глобальной/файловой областью видимости** доступны в любом месте программы/файла. Это:
  - глобальные переменные.

## Продолжительность жизни

**Продолжительность жизни** переменной определяет, где она создается и где уничтожается.

- Переменные с **автоматической продолжительностью жизни** создаются в точке определения и уничтожаются при выходе из блока, в котором определены. Это:
  - обычные локальные переменные.
- Переменные со **статической продолжительностью жизни** создаются, когда программа запускается, и уничтожаются при её завершении. Это:
  - глобальные переменные;
  - статические локальные переменные.
- Переменные с **динамической продолжительностью жизни** создаются и уничтожаются по запросу программиста. Это:
  - динамические переменные (о них мы поговорим на соответствующем уроке).

## Связи

**Связь** идентификатора определяет, относятся ли несколько упоминаний одного идентификатора к одному и тому же идентификатору или нет.

- Идентификаторы **без связей** — это идентификаторы, которые ссылаются сами на себя. Это:
  - обычные локальные переменные;
  - пользовательские типы данных, такие как enum, typedef и классы, объявленные внутри блока (об этом детально поговорим на соответствующих уроках).
- Идентификаторы с **внутренней связью** доступны в любом месте файла, в котором они объявлены. Это:
  - статические глобальные переменные (инициализированные или неинициализированные);
  - константные глобальные переменные;
  - статические функции (о них поговорим чуть позже).
- Идентификаторы с **внешней связью** доступны как в любом месте файла, в котором они объявлены, так и в других файлах (через предварительное объявление). Это:
  - обычные функции;
  - неконстантные глобальные переменные (инициализированные или неинициализированные);
  - внешние константные глобальные переменные;
  - определяемые пользователем типы данных, такие как enum, typedef и классы с глобальной областью видимости (о них мы поговорим чуть позже).

Идентификаторы с внешней связью могут вызвать ошибку дублирования определений, если определения скомпилированы в более чем одном файле .cpp.

Функции по умолчанию имеют внешнюю связь, что можно изменить с помощью ключевого слова static (на внутреннюю связь).

Внимательные читатели могут заметить, что глобальные типы данных имеют внешнюю связь, но их определения не вызывают ошибки линкера при использовании в нескольких файлах. Это связано с тем, что типы, шаблоны и внешние встроенные функции являются исключениями из правила, и это позволяет им быть определенными более чем в одном файле, при условии, что эти определения идентичны. В противном случае, они не были бы так полезны.

## Резюмируем

Весь материал, изложенный выше:

| Тип | Пример | Область видимости | Продолжительность жизни | Связь | Примечание |
|---|---|---|---|---|---|
| **Локальная переменная** | int x; | Локальная область видимости | Автоматическая продолжительность жизни | Нет связей | |
| **Статическая локальная переменная** | static int s_x; | Локальная область видимости | Статическая продолжительность жизни | Нет связей | |
| **Динамическая переменная** | int *x = new int; | Локальная область видимости | Динамическая продолжительность жизни | Нет связей | |
| **Параметр функции** | void foo(int x) | Локальная область видимости | Автоматическая продолжительность жизни | Нет связей | |
| **Внешняя неконстантная глобальная переменная** | int g_x; | Глобальная область видимости | Статическая продолжительность жизни | Внешняя связь | Инициализированная или неинициализированная. |
| **Внутренняя неконстантная глобальная переменная** | static int g_x; | Глобальная область видимости | Статическая продолжительность жизни | Внутренняя связь | Инициализированная или неинициализированная. |
| **Внутренняя константная глобальная переменная** | const int g_x(1); | Глобальная область видимости | Статическая продолжительность жизни | Внутренняя связь | Должна быть инициализирована. |
| **Внешняя константная глобальная переменная** | extern const int g_x(1); | Глобальная область видимости | Статическая продолжительность жизни | Внешняя связь | Должна быть инициализирована. |

## Предварительные объявления

С помощью предварительного объявления мы можем получить доступ к функции или переменной из другого файла:

| Тип | Пример | Примечание |
|---|---|---|
| **Предварительное объявление функции** | void foo(int x); | Только прототип, без тела функции. |
| **Предварительное объявление неконстантной глобальной переменной** | extern int g_x; | Переменная **НЕ** должна быть инициализирована. |
| **Предварительное объявление константной глобальной переменной** | extern const int g_x; | Переменная **НЕ** должна быть инициализирована. |